

## WHAT THIS SESSION IS

Run on 6 September 2026, and it is one topic done properly. By the end of the last class the agent was a closed box: model plus tools plus loop, all handled by `create_agent`. That is convenient right up to the moment you need to change the system prompt per customer, hide half your tools from the model, retry a tool that failed, keep a long conversation inside budget, or send the hard questions to an expensive model and the easy ones to a cheap one. Middleware is how you do all of that without touching the agent's own work. Seven files: a baseline with none, the four hooks demonstrated together, three custom middlewares you write yourself, and two built into the framework.

## CONTENTS

1. Where this session picks up
2. What middleware is, and the security guard
3. The baseline: an agent with no middleware
4. The four hooks, and where each one sits
5. All four hooks in one file
6. Reading the output, and a surprise in it
7. A system prompt that changes per customer
8. Choosing which tools the model may even see
9. Built in: retrying a tool that failed
10. Built in: summarising a long conversation
11. Routing the hard questions to a stronger model
12. Which hook should you reach for
13. Clarifications from the Q&A
14. Key takeaways

## 1 Where this session picks up

Last class produced an agent that works: it calls the model, runs whatever tools the model asks for, loops until there is an answer, and remembers a conversation if you give it a checkpoint. All of that happens inside `create_agent`.

Now the question is control. You do not want to change what the agent does. You want to change what happens *around* it.

#### A WORD ON PRACTICE, SAID AT THE START

Watching without practising was called out directly: within the first month you will fall behind and only notice later. The code for every session is on the dashboard under the session's resources, alongside the notes and the board work. Run it yourself.

## 2 What middleware is, and the security guard

The analogy used in class is the clearest way in.

Think of an office. There is an **employee** who does the actual work, and there is a **security guard** at the door. The guard does not do the employee's job and does not get in the way of it. But the guard checks who is entering, can turn somebody away, logs what happened, and applies the rules.

The employee is the agent. The guard is the middleware. And crucially, the employee does not need to carry any of the security logic, because that lives with the guard.

## WITHOUT MIDDLEWARE

```
user question
  |
  |
  v
+-----+
| AGENT |
| model |
|  +    |
| tools |
|  +    |
| loop  |
+-----+
  |
  v
answer
```

## WITH MIDDLEWARE

```
user question
  |
[ middleware ] ← inspect, modify,
                even stop
  |
  v
+-----+
| AGENT |
| model |
|  +    |
| tools |
|  +    |
| loop  |
+-----+
  |
[ middleware ] ← check the output
                before it leaves
  |
  v
answer
```

the agent's own work is unchanged in both.  
the layer around it is what you are adding.

## A concrete example, from last week's code

A customer types *refund my order*. Last week that went straight to the agent, the model decided a refund tool was needed, and the refund started.

With middleware sitting in front of the tool call you can first ask: is this user authenticated? Is that a real order? Has a refund already been processed for it? Is the amount higher than the order itself? Only if all of that passes does the tool actually run.

### IT IS NOT ONLY FOR SECURITY

Stated more than once, because the refund example makes people think guardrails. Middleware is anything you want to happen around the agent. This session alone uses it to build a prompt per customer, filter the tool list, retry a failure, compress a conversation and switch models. Not one of those is a security check.

Related, and asked in the room: it is **not** an API gateway, though routing is one thing you can do with it. It is closer in spirit to aspect oriented programming, where the supporting logic is kept out of the main business logic. The trainer accepted that comparison as "something similar", not exact.

Reference: [docs.langchain.com/oss/python/langchain/middleware](https://docs.langchain.com/oss/python/langchain/middleware)

### WHY NOT JUST PUT THIS LOGIC INSIDE THE TOOLS

Asked by several people, and it is a fair question. You can. Authentication, logging, masking personal data, token monitoring, approval checks: all of it could go inside each tool.

Then you have written it once per tool, and it is tangled up with the actual work. Middleware keeps it in one place, applied to everything, and leaves your tools doing only what they are for.

## 3 The baseline: an agent with no middleware

The session opens with the same agent as last week, so there is something to compare against. Nothing here is new.

## nomiddleware.py

```
import sys

from dotenv import load_dotenv
from langchain.agents import create_agent
from langchain_core.tools import tool

load_dotenv()
sys.stdout.reconfigure(encoding="utf-8")

ORDERS = {
    "ORD-1001": {"item": "Wireless mouse", "status": "shipped", "amount": 1499},
    "ORD-1002": {"item": "Mechanical keyboard", "status": "packed", "amount": 4999},
}

@tool
def order_status(order_id: str) → str:
    """Get the item, status and amount of an order using its id."""
    order = ORDERS.get(order_id.upper())
    if order is None:
        return f"No order found with id {order_id}."
    return f"{order['item']}, status {order['status']}, amount {order['amount']} rupees"

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[order_status],
    system_prompt="You are a support agent. Look up the order before answering.",
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Where is my order ORD-1002?"}]
})

print(result["messages"][-1].content)
```

It runs, it looks up the order, it answers. There is no `middleware=` parameter anywhere. Every file after this one adds exactly that.

## Project setup

A fresh uv project, the same as always, with LangGraph and Pydantic now explicitly declared because the agent and the built-in middlewares need them:

```
uv init
uv venv
.venv\Scripts\activate          # macOS and Linux: source .venv/bin/activate

uv add langchain langchain-openai langgraph pydantic python-dotenv requests
```

### ON THE API KEY, SINCE IT COMES UP EVERY WEEK

The trainer creates a fresh key before each class and revokes it afterwards, because the key is visible on screen and the code files get shared. That is his situation, not a rule. You do not need a new key per day or per project. What you do need is to keep it in `.env`, keep `.env` out of Git, and revoke anything that leaks.

## 4 The four hooks, and where each one sits

Middleware attaches to the agent loop at four points. Every custom middleware in this session is one of these four.

```

agent.invoke(...)
|
v
+-----+
| @before_model | runs BEFORE the model is called
| inspect or change | sees the state, can change it
| the state | return None = change nothing
+-----+
|
+-----v-----+
| @wrap_model_call | sits AROUND the model call
|
| code before .....|..... your logic
| handler(request) --|→ THE MODEL RUNS HERE
| code after .....|..... your logic
+-----+
|
+-----v-----+
| @after_model | runs AFTER the model replies
| see what it asked |
| for, or the answer |
+-----+
|
did it ask for tools?
|
+-----v-----+
| @wrap_tool_call | sits AROUND every tool run
|
| code before .....|..... check or change the input
| handler(request) --|→ THE TOOL RUNS HERE
| code after .....|..... check or change the output
+-----+
|
+---- back to the model, until it stops asking

```

| Hook                          | Runs                    | Use it when                                                                                               |
|-------------------------------|-------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>@before_model</code>    | Before the model call   | You want to inspect or change the state, for example adding information to the conversation               |
| <code>@wrap_model_call</code> | Around the model call   | You need the request object itself: swap the model, filter the tools, time the call, inspect the response |
| <code>@after_model</code>     | After the model replies | You want to see or change what came back                                                                  |
| <code>@wrap_tool_call</code>  | Around every tool run   | Validate the tool input, change the output, log, handle errors, retry                                     |

**THE ONE LINE THAT APPEARS IN EVERY WRAP HOOK**

`handler(request)` is the pivot. Everything above it runs before, everything below it runs after, and the line itself is *the actual thing happening*: the model being called, or the tool being run.

It was asked whether `handler` is the same as `invoke`. It is not. `invoke` starts the whole agent. `handler` means "my part is done, carry on with the next step". The flow is: you call `agent.invoke`, which calls the middleware, which calls the handler, which calls the model.

**5 All four hooks in one file**

This file does nothing useful on purpose. Every hook only prints, so you can watch the order they fire in. Real logic starts in the next section.

## middleware-basic1.py

```
from langchain.agents import create_agent
from langchain.agents.middleware import (
    after_model,
    before_model,
    wrap_model_call,
    wrap_tool_call,
)
from langchain_core.tools import tool

@tool
def order_status(order_id: str) → str:
    """Get the status of an order using its id."""
    return f"{order_id.upper()}: packed, ships tomorrow"

# runs before the model is called
@before_model
def show_before(state, runtime):
    print(f"[before_model] {len(state['messages'])} message(s) going to the model")
    return None # None means: I am not changing the state

# around the model call
@wrap_model_call
def time_the_model(request, handler):
    print(f"[wrap_model_call] tools offered: {[t.name for t in request.tools]}")
    response = handler(request) # ← the model runs here
    print("[wrap_model_call] model has replied")
    return response

# runs immediately after the model responds
@after_model
def show_after(state, runtime):
    last = state["messages"][-1]
    asked = [call["name"] for call in getattr(last, "tool_calls", []) or []]
    print(f"[after_model] asked for: {asked or 'nothing, this is the final answer'}")
    return None
```

```

# sits around every tool execution
@wrap_tool_call
def show_tool(request, handler):
    print(f"[wrap_tool_call] running {request.tool_call['name']} "
          f"with {request.tool_call['args']}")
    result = handler(request)          # ← the tool runs here
    print("[wrap_tool_call] tool finished")
    return result

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[order_status],
    system_prompt="You are a support agent.",
    middleware=[show_before, time_the_model, show_after, show_tool],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Where is my order ORD-1002?"}]
})
print()
print("Answer:", result["messages"][-1].content)

```

## What each one is actually looking at

- **state** in the before and after hooks holds the conversation. `state["messages"]` is the message list you already know from last session, so `len(...)` is how many messages are about to be sent.
- **request.tools** in `wrap_model_call` is the list of tools that will be offered to the model on this call. That single fact is what makes section 8 possible.
- **request.tool\_call** in `wrap_tool_call` holds the name and the arguments the model chose. That is where you would check the input, because those arguments came from the LLM.
- **return None** in the before and after hooks means "I am only observing, change nothing". The wrap hooks must return the response or result instead, because they sit in the middle of the flow.
- **middleware=[...]** is the only change to `create_agent`. Four middlewares here, and there is no limit on how many you attach.

## 6 Reading the output, and a surprise in it

```
[before_model]    1 message(s) going to the model
[wrap_model_call] tools offered: ['order_status']
[wrap_model_call] model has replied
[after_model]     asked for: ['order_status']

[wrap_tool_call]  running order_status with {'order_id': 'ORD-1002'}
[wrap_tool_call]  tool finished

[before_model]    3 message(s) going to the model      ← AGAIN
[wrap_model_call] tools offered: ['order_status']      ← AGAIN
[wrap_model_call] model has replied
[after_model]     asked for: nothing, this is the final answer

Answer: ORD-1002 is packed and ships tomorrow.
```

### MIDDLEWARE RUNS PER MODEL CALL, NOT PER USER QUESTION

This caught several people out, and it was asked repeatedly in the chat. The hooks fired twice. That is not a bug.

Recall the loop from last session. One user question that needs a tool means **two** trips to the model: once to decide the tool, and once again after the tool result comes back so it can write the answer. The middleware wraps *every* model call, so it runs on both.

It matters more than it looks. If your middleware costs something, filters tools, or picks a model, it is doing that on every step of the loop, not once at the door.

The message count going from 1 to 3 is the same story: the second call carries the original question, the model's tool request and the tool result.

## 7 A system prompt that changes per customer

Every system prompt so far has been fixed. One string, written once, identical for everybody who uses your application.

That is not how a human support agent behaves. You speak differently to a premium customer, to somebody's first ever query, and to a customer who only reads Hindi. The `@dynamic_prompt` hook lets the agent do the same.

## The context object

```
from dataclasses import dataclass

@dataclass
class Customer:
    """What we know about the person asking, before the model sees anything."""

    name: str
    plan: str
    language: str
```

A `dataclass` is a Python shortcut for a class that mainly holds data, without writing the boilerplate. Nothing AI specific about it.

### THIS INFORMATION DOES NOT COME FROM THE USER'S MESSAGE

An important distinction made in class. The customer never types "I am a premium customer". Your application already knows that, from your database or the session. Context is what your application knows; messages are what the user said. Middleware is where the two meet.

## dynamic-prompt-mw.py

```
from langchain.agents.middleware import dynamic_prompt

@dynamic_prompt
def support_prompt(request):

    # gives us the Customer object we passed in at agent.invoke()
    customer = request.runtime.context

    lines = [
        f"You are a support agent for an online store. The customer is {customer.name}.",
        "Look up the order before answering, never guess.",
        "Write like a chat reply, no email signature.",
    ]

    if customer.plan == "premium":
        lines.append("This is a premium customer, apologise for any delay and offer a callback.")
    else:
        lines.append("This is a free plan customer, keep the answer to two lines.")

    lines.append(f"Reply in {customer.language}.")

    prompt = " ".join(lines)
    print("[prompt used]", prompt)
    print()
    return prompt

agent = create_agent(
    model="gpt-4o-mini",
    tools=[order_status],
    middleware=[support_prompt],
    context_schema=Customer,          # tells the agent what kind of context to expect
)

question = {"messages": [{"role": "user", "content": "Where is my order ORD-1002?"}]}
```

  

```
for customer in [
    Customer(name="Asha", plan="premium", language="English"),
```

```

Customer(name="Ramesh", plan="free", language="Hindi"),
]:
    result = agent.invoke(question, context=customer)
    print(f"To {customer.name} ({customer.plan}):")
    print(result["messages"][-1].content)

```

## Three pieces that have to line up

1. context\_schema=Customer                      on create\_agent  
"an object of this type will arrive at runtime"
  2. context=customer                              on agent.invoke  
the actual object, for this one call
  3. request.runtime.context                      inside the middleware  
reaching in and reading it
- miss any one of the three and the other two do nothing.

## Same question, same agent, two different answers

|                                |                               |
|--------------------------------|-------------------------------|
| ASHA, premium, English         | RAMESH, free, Hindi           |
| prompt built for her:          | prompt built for him:         |
| "...the customer is Asha...    | "...the customer is Ramesh... |
| this is a premium customer,    | this is a free plan customer, |
| apologise for any delay and    | keep the answer to two lines. |
| offer a callback.              | Reply in Hindi."              |
| Reply in English."             |                               |
| reply:                         | reply:                        |
| apologetic, offers a callback, | short, two lines,             |
| in English                     | in Hindi                      |

ONE agent. ONE question. The prompt was built per request.

### HONEST STATUS FROM THE LIVE CLASS

The file failed on the first run. The parameter had been typed as `context_type` instead of `context_schema`, an autocomplete suggestion that looked right and was not. Changing it fixed the run immediately.

Worth noticing that the error pointed at the `create_agent` line, and several minutes went into checking the model name before the actual cause was found. When a constructor rejects something, read the parameter names before you doubt the values.

### WHY A DEDICATED HOOK WHEN BEFORE\_MODEL EXISTS

Asked well, and answered clearly. You *could* do this with `@before_model`. `@dynamic_prompt` exists for exactly one job, building the system prompt, so using it says what you mean.

It also runs before the model, and if you had both a `@dynamic_prompt` and a `@wrap_model_call`, the dynamic prompt goes first. Rule of thumb given in class: use `@before_model` when you are inspecting or changing state, use `@dynamic_prompt` when you are building a prompt, and use `@wrap_model_call` for everything else.

## 8 Choosing which tools the model may even see

By default every tool you registered goes to the model on every single call. With four tools that is fine. With fifty it is a problem twice over.

- **Cost.** Every tool's name, description and argument schema is sent in the request, every time. Fifty tools is a meaningful number of tokens on every step of the loop.
- **Accuracy.** If several tools do similar things, the model has more chances to pick the wrong one. More options is not more capable.

### SAY WHAT THIS IS NOT

Stated explicitly in class: this is **not** a security mechanism. It is about giving the model the right tools at the right time. A user cannot invoke a hidden tool, but do not treat tool filtering as your authorisation layer.

## Grouping the tools

```
DELIVERY_TOOLS = [order_status, delivery_estimate]

MONEY_TOOLS    = [order_status, start_refund, cancel_order]

MONEY_WORDS    = ("refund", "cancel", "money", "return")
```

Note that `order_status` appears in both groups. That was deliberate and the trainer pointed it out: a money question often still needs to look the order up first.

## Finding the customer's actual question

```
def latest_question(messages):
    """
    Get the customer's most recent message.

    We don't simply use messages[-1]. During an agent run the last
    message could be a tool result, the model's response, or another
    internal message. We specifically want the latest human message.
    """
    for message in reversed(messages):
        if type(message).__name__ == "HumanMessage":
            return str(message.content).lower()

    return ""
```

This helper matters more than it looks, and it comes back again in section 11. Halfway through a loop the last message is very often a `ToolMessage`, so `messages[-1]` would be reading the wrong thing. Walking backwards until you hit a `HumanMessage` is the reliable way.

## dynamic-tool-mw.py

```
from langchain.agents.middleware import wrap_model_call

@wrap_model_call
def pick_tools(request, handler):

    question = latest_question(request.messages)

    if any(word in question for word in MONEY_WORDS):
        request = request.override(tools=MONEY_TOOLS)
    else:
        request = request.override(tools=DELIVERY_TOOLS)

    print("    offered:", [t.name for t in request.tools])

    return handler(request)

agent = create_agent(
    model="gpt-4o-mini",
    tools=[order_status, delivery_estimate, start_refund, cancel_order],
    middleware=[pick_tools],
    system_prompt="You are a support agent. Use the tools, never guess.",
)
```

Q: "Where is my order ORD-1002?"

no money word found

offered: ['order\_status', 'delivery\_estimate'] 2 of 4

Q: "I want a refund for ORD-1002, it is too late for me."

"refund" matched

offered: ['order\_status', 'start\_refund', 'cancel\_order'] 3 of 4

the agent was built with four tools.

the model never sees all four at once.

### REQUEST.OVERRIDE(...) IS THE WHOLE TRICK

The request object is what is about to be sent to the model. `override` returns a modified copy of it, and then you hand *that* to the handler. Change the tool list and the model is offered a different set. Change the model, as in section 11, and a different model answers.

Everything you can override in `wrap_model_call` is a lever, and this is the hook that gives you the request in the first place. That is why it is the one you reach for most.

### ON THE KEYWORD MATCHING

Matching on four hard-coded words is fine for a console demo and was called out as such. In a real application you would classify the request properly rather than listing money-related words by hand. The mechanism is the lesson here, not the heuristic.

## 9 Built in: retrying a tool that failed

Everything so far has been a **custom** middleware: you wrote the function, you chose the hook. The framework also ships middlewares where the logic is already written.

A tool can fail for reasons that have nothing to do with your code: a network blip, an API outage, a timeout, an external service having a bad minute. You do not want the agent to give up on the first failure.

```
from langchain.agents.middleware import ToolRetryMiddleware

attempts = {"count": 0}

@tool
def order_status(order_id: str) → str:
    """Get the status of an order using its id."""

    attempts["count"] += 1
    print(f"    attempt {attempts['count']} to reach the order service")

    # deliberately fail the first two attempts, to simulate an outage
    if attempts["count"] < 3:
        raise RuntimeError("order service unavailable")

    return f"{order_id.upper()}: packed, ships tomorrow"

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[order_status],
    system_prompt="You are a support agent. Look up the order before answering.",
    middleware=[
        ToolRetryMiddleware(
            max_retries=5,
            initial_delay=0.2,
            backoff_factor=1.5
        )
    ],
)

result = agent.invoke({
    "messages": [{"role": "user", "content": "Where is my order ORD-1002?"}]
})

print("Answer:", result["messages"][-1].content)
print("Total attempts:", attempts["count"])
```

```
attempt 1 to reach the order service    → raises, wait 0.2s
attempt 2 to reach the order service    → raises, wait 0.3s
attempt 3 to reach the order service    → succeeds
```

Answer: ORD-1002 is packed and ships tomorrow.

Total attempts: 3

the three settings:

```
max_retries    = 5    how many extra tries after the first failure
initial_delay  = 0.2   how long to wait before the first retry
backoff_factor = 1.5   multiply the wait by this each time,
                        so the gaps grow instead of hammering
                        a service that is already struggling
```

### CUSTOM AGAINST BUILT IN

Notice what is missing from this file. No `@wrap_tool_call`, no function of your own, no `handler(request)`. You import a class and pass it in.

Which is the trade. Built-in middlewares are less code and battle tested; custom middlewares do exactly what you decide. Everything in sections 7, 8 and 11 is custom because no built-in matches it. Retrying is generic enough that somebody has already written it well.

Anyone from a microservices background will recognise this as the retry half of the retry and circuit breaker pattern, which was mentioned in the room.

## 10 Built in: summarising a long conversation

This one solves a problem that has been building since session 2.

Every call sends the whole conversation. Turn ten sends nine previous exchanges plus the new question. That costs tokens, and eventually it hits the **context window**, the maximum the model can hold at once. Past that limit the earliest messages fall off, and the model starts forgetting the beginning of the conversation.

## WITHOUT SUMMARISATION

```
call 1  [ m1 ]  
call 2  [ m1 m2 m3 ]  
call 3  [ m1 m2 m3 m4 m5 ]  
call 4  [ m1 ... m7 ]  
call 5  [ m1 ... m9 ]  
...  
call 40 [ m1 ..... m79 ]
```

the old messages are re-sent, in full, on every single call.

you pay for m1 forty times.

## WITH SummarizationMiddleware

```
trigger = 6 messages  
keep    = the latest 2  
  
once the history reaches 6:  
  
[ m1 m2 m3 m4 ] → one summary  
[ m5 m6 ]       → kept as they are
```

[ summary ][ m5 ][ m6 ][ new ]

far fewer tokens, and the important facts survive.

## summary-mw.py

```
from langchain.agents.middleware import SummarizationMiddleware

agent = create_agent(
    model="openai:gpt-4o-mini",
    tools=[],
    system_prompt="You are a support agent for an online store.",
    middleware=[
        SummarizationMiddleware(
            model="openai:gpt-4o-mini", # which model writes the summary
            trigger=("messages", 6),    # summarise once history reaches 6 messages
            keep=("messages", 2)        # keep the latest 2 untouched
        )
    ],
)

conversation = [
    "Hi, my name is Asha and my customer id is C-9087.",
    "I ordered a mechanical keyboard last week, order ORD-1002.",
    "My pin code is 560034, in Bengaluru.",
    "I also have an older order, ORD-1001, a wireless mouse.",
    "The keyboard is a gift, so the date matters to me.",
    "Tell me everything you remember about me and my orders.",
]

messages = []

for question in conversation:
    messages.append({"role": "user", "content": question})
    result = agent.invoke({"messages": messages})
    messages = result["messages"] # carry the history forward, summarised or not

    print("Customer:", question)
    print("Agent   :", result["messages"][-1].content[:160])
    print(f"   history now holds {len(messages)} messages")
```

Run live across six turns, the history grew normally at first, then stopped growing. Printing the full message list showed the early exchanges replaced by a single summary that still contained Asha's name, her customer id and both order numbers, with the two most recent messages intact.

### THE BEST QUESTION OF THE SESSION

A student asked: summarising needs a model call of its own, so does this not just cost tokens too?

Yes, and it still wins. Without summarisation you re-send every old message on **every** call, so message one is paid for again on call two, call three, call forty. The summary is produced **once** and then that short text is what travels from then on.

You can also summarise with a cheaper model than the one answering. The `model` parameter is separate for exactly that reason, and a small local model is a perfectly reasonable choice for it.

### TWO THINGS THIS FILE IS NOT DOING

**No thread id.** Unlike last session there is no checkpoint here, and the history is carried in a plain Python variable. This is a single-user demonstration, and it was clarified when somebody asked. In a real application you would combine a checkpoint with summarisation.

**Can you summarise without an LLM?** Asked, and the answer was effectively no. You could write your own compression rules, but you cannot condense natural language and keep the meaning without a model. That is what a model is for.

Also worth knowing: the **context window is not something you set**. It is a property of the model, decided by the provider. Your side of it is deciding what to send.

## 11 Routing the hard questions to a stronger model

The last example changes the model itself, per request. Easy questions go to the cheap model. Hard ones go to the strong one.

```

from langchain.chat_models import init_chat_model
from langchain.agents.middleware import wrap_model_call

CHEAP = init_chat_model("openai:gpt-4o-mini")
STRONG = init_chat_model("openai:gpt-4o")

HARD_WORDS = (
    "hard", "difficult", "complex", "complicated", "challenging",
    "compare", "explain why", "analyze", "risk", "legal"
)

@wrap_model_call
def route_model(request, handler):

    question = latest_question(request.messages)

    hard = (
        any(word in question for word in HARD_WORDS)
        or len(question) > 180
    )

    chosen = STRONG if hard else CHEAP

    print(f"    routed to {chosen.model_name}")

    return handler(request.override(model=chosen))

agent = create_agent(
    model=CHEAP,                # the default, used unless the middleware overrides
    tools=[order_status],
    system_prompt="You are a support agent for an online store.",
    middleware=[route_model],
)

ask("Where is my order ORD-1002?")
ask("Compare buying a mechanical keyboard now against waiting for the sale, "
    "and explain why.")

```

Q: "Where is my order ORD-1002?"

no hard word, short question

routed to gpt-4o-mini the cheap one

Q: "Compare buying a mechanical keyboard now against

waiting for the sale, and explain why."

"compare" and "explain why" both matched

routed to gpt-4o the expensive one

two criteria: a keyword list, and length over 180 characters,  
on the theory that a long question is usually a harder one.

### HONEST STATUS FROM THE LIVE CLASS

The first run always printed `gpt-4o-mini`, even for the hard question. The last line had been left as plain `handler(request)`, taken from an autocomplete suggestion. It chose a model, printed the choice, and then sent the original unmodified request.

The fix is the point of the whole file: `handler(request.override(model=chosen))`. Deciding something in middleware does nothing until you put the decision back into the request. Without the override, this example does not work at all.

### WHY THIS IS WORTH DOING

Frontier models cost several times more per token than the small ones, and most support questions do not need one. Routing the easy majority to the cheap model and reserving the strong one for the genuinely hard minority is a straightforward saving, and the caller never notices.

It is the same pressure that pushes companies towards open weights models and their own GPU infrastructure, which came up again here. Token cost at scale is a real line item, not a detail.

### TWO DECISIONS, NOT ONE

A good clarification from the Q&A. Your middleware decides **which model handles the question**. The model it picks then reads the question and decides **what to answer**. Routing is not answering, and the router never looks at the content beyond matching a few words.

## 12 Which hook should you reach for

| You want to                                  | Hook                                                          | Seen in    |
|----------------------------------------------|---------------------------------------------------------------|------------|
| Build the system prompt per request          | <code>@dynamic_prompt</code>                                  | Section 7  |
| Inspect or change the state before the model | <code>@before_model</code>                                    | Section 5  |
| Change the tools offered                     | <code>@wrap_model_call</code> + <code>request.override</code> | Section 8  |
| Change which model answers                   | <code>@wrap_model_call</code> + <code>request.override</code> | Section 11 |
| See or change what the model returned        | <code>@after_model</code>                                     | Section 5  |
| Validate or change tool input and output     | <code>@wrap_tool_call</code>                                  | Section 5  |
| Retry a failing tool                         | <code>ToolRetryMiddleware</code>                              | Section 9  |
| Keep a long conversation in budget           | <code>SummarizationMiddleware</code>                          | Section 10 |

#### THE RULE OF THUMB GIVEN IN CLASS

If you are **inspecting or changing the state**, use `@before_model`. If you are **building a system prompt**, use `@dynamic_prompt`. For everything else, especially anything that needs the request object itself, use `@wrap_model_call`.

And you are not limited to one. The basics file used four at once, and there is no cap.

## 13 Clarifications from the Q&A

- **Why did the middleware print twice for one question?** Because it runs per model call. A question that uses a tool means two model calls: one to choose the tool, one to write the answer after seeing the result.
- **Is `handler(request)` the same as `invoke`?** No. `invoke` starts the whole agent. `handler` continues the current step. The order is: `agent.invoke`, then middleware, then handler, then the model.
- **Could I put all this logic inside the tools instead?** Yes, and it gets messy fast. Middleware keeps supporting logic in one place, applied everywhere, and leaves the tools doing only their job.

- **Is middleware the same as guardrails?** Related but not the same. Guardrails are about what the model is allowed to receive and produce, with input and output guardrails as separate ideas. Middleware is the general mechanism, and guardrails are one thing you can build with it. Guardrails get their own sessions.
- **Can I use more than one middleware?** Yes, no limit.
- **If both `@dynamic_prompt` and `@wrap_model_call` exist, which runs first?** The dynamic prompt, since it exists specifically to prepare the prompt before the model call.
- **Can I use a different model for summarisation than for answering?** Yes, and you probably should. A cheaper or local model is fine for compressing a conversation.
- **Where is the context window limit set?** Not by you. It is a property of the model, decided by the provider.
- **Does the summariser handle both user and model messages?** The whole conversation, including tool results.
- **Can we use this in production?** Yes. The trainer noted Telusko runs mostly open weights models on its own GPU infrastructure, and the same middleware code applies either way.
- **My local Llama model passed the wrong arguments to a tool. Why?** Smaller models are less reliable at tool calling and argument generation. It is a model capability issue, not a code issue.
- **Which hook is used most in production?** There is no single answer. It depends entirely on the application's requirements and architecture.
- **What if the agent needs a calculation and no tool provides it?** The model will still answer, and it may hallucinate. If the exact figure is not in the context and no tool can produce it, you are relying on prediction. Add the tool.

## 14 Key takeaways

- **Middleware is the security guard, the agent is the employee.** It controls what happens around the agent without touching the agent's own work.
- **Four hooks:** `@before_model`, `@wrap_model_call`, `@after_model`, `@wrap_tool_call`. Everything custom is one of these.
- **`handler(request)` is the pivot in every wrap hook.** Above it runs before, below it runs after, and the line itself is the model or the tool actually running.
- **Middleware runs per model call, not per question.** One tool-using question fires it twice.
- **`request.override(...)` is how a decision takes effect.** Choosing a model or a tool list does nothing until you override the request and pass that on.
- **`@dynamic_prompt` plus `context_schema` plus `context=`** gives you a system prompt built per customer. All three pieces are required.
- **Filtering tools saves tokens and improves accuracy,** because fifty similar tools make the model's choice harder, not easier. It is not a security control.
- **Walk backwards for the latest human message.** Mid-loop, `messages[-1]` is often a tool result.

- **Custom middleware is a function you write; built-in middleware is a class you import.**  
`ToolRetryMiddleware` and `SummarizationMiddleware` are both built in.
- **Retry with backoff** makes an agent resilient to a service having a bad minute.
- **Summarisation costs one model call and saves it many times over**, because the alternative is re-sending the whole history on every single call.
- **Route easy questions to a cheap model and hard ones to a strong model.** The caller never notices, and the bill does.

#### COMING NEXT

A few more middleware examples and a slightly larger application that puts them together, then **MCP**, which may be led by a different instructor. After that the roadmap stays as stated: RAG so the agent can search your own documents, human in the loop so it pauses for approval before something risky, then evaluation, tracing and guardrails. Assignments were promised before MCP begins.

---

Telusko | Agentic AI Engineering with Python. Notes for Session 06, recorded 6 September 2026.